

Meru मेरु

How it's built, and how to build it with us.

In some Eastern traditions, Meru is the cosmic mountain that the sun, moon and stars turn around.

v0.7.4 · for developers · November 2026

github.com/aarora79/meru
Scan for the code

THE PROJECT

A local AI assistant, written in Go, built in the open

Two programs

`merud`, a daemon that keeps models warm and does the work, and `meru`, a thin client over a Unix socket.

Three clients

`meru "…"` for one question, `meru chat` in a terminal, and Meru.app, a Wails v3 window.

Local models

Ollama on loopback runs the router and search models always, and the answer model unless you name one on your own network.

Why Go

Single native binaries with no cgo in `merud`, cheap goroutines for a daemon, and code a Python developer can read.

The size

34 internal packages, 6 commands, a coding note per package and 31 notes on Go itself.

The licence

Apache 2.0. Nothing for sale, anywhere.

HOW WE DECIDE

Simple wins every time

- When two designs both work, pick the one with fewer moving parts.
- A function beats a type. A struct beats an interface. An interface appears with its second implementation.
- One way to do each thing: one `dispatch` path, one store, one config file.
- No speculative flags, options or plugin points.
- The standard library first, boring libraries second.
- Add complexity only when a measurement or a second real use asks for it, and say which in the PR.
- Write for a reader new to Go: every package, file and function explains itself.
- `ARCHITECTURE.md` is the contract; change it before the code.

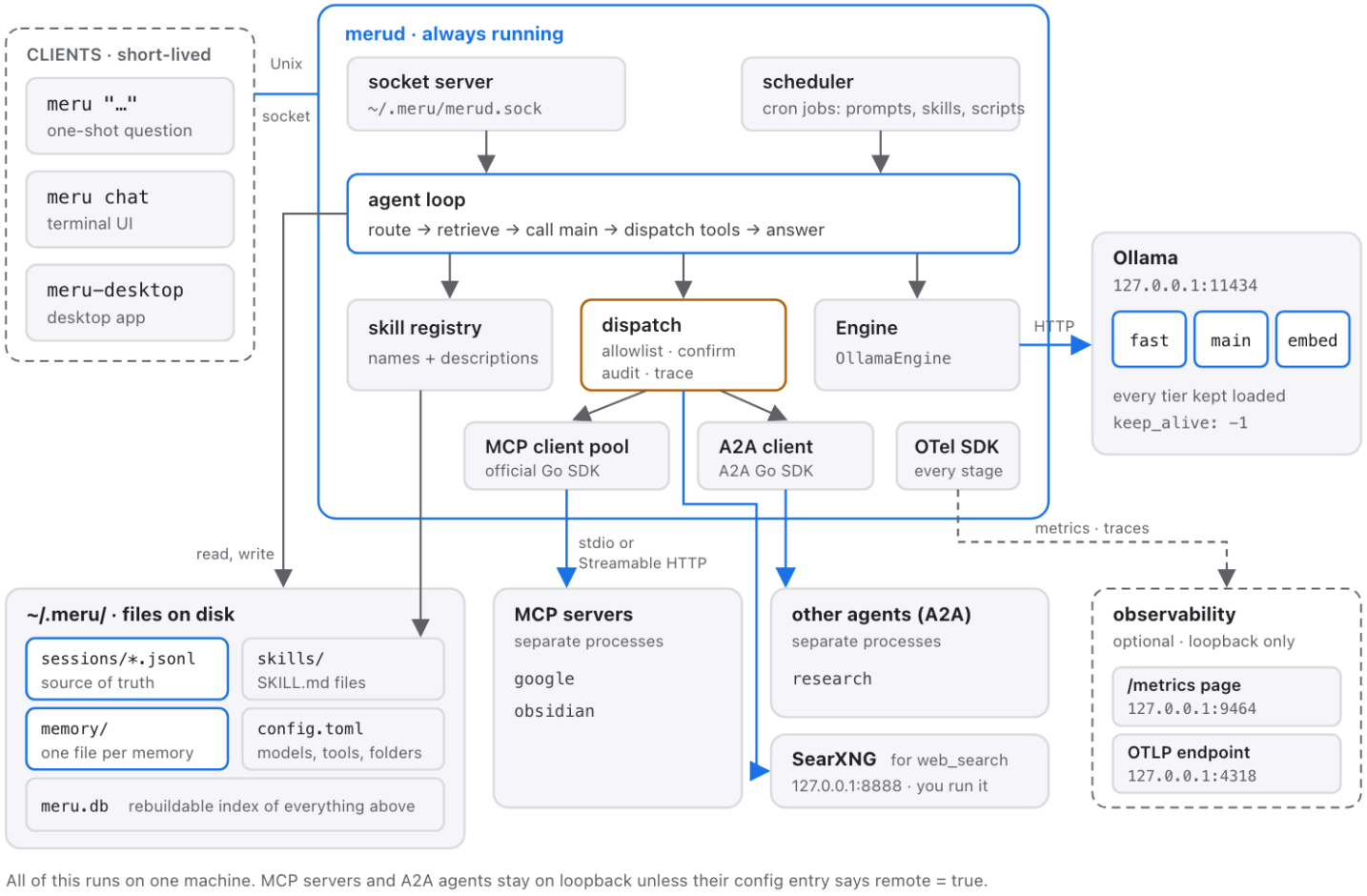
THE RULES THAT DON'T BEND

Five non-negotiables, enforced in CI

Rule	What it means in code
No prompt leaves without a yes	Only an answer model you name may run on your own network, after "this time" or "this chat". No provider SDK or address in code.
One anonymous daily count, no other telemetry	Seven fixed fields with no ID, sent only by `internal/usagereport` after a client shows the notice. OpenTelemetry exports to loopback only; no update checks, no crash reports.
Deny by default	A new MCP server or A2A agent offers nothing until config allowlists it.
Every tool call through `dispatch`	One path logs each call to `tool_calls` and the transcript. Never a second one.
Nothing for sale	No pricing, paid licence or sponsor link anywhere in the project.

`internal/policy` holds the tests: deny-lists of hosts and imports, allowed URLs, and the thin-client rules.

ARCHITECTURE



All of this runs on one machine. MCP servers and A2A agents stay on loopback unless their config entry says remote = true.

A daemon, thin clients and files you own

- `merud` owns the models, the store, the tools and the agent loop.
- Clients hold no model or store logic, and policy tests keep it so.
- Files under `~/.meru` are the truth; `meru.db` is a rebuildable index.

THE HARNESS

One question, end to end, in `merud`

1

Request

One JSON line on the Unix socket: the question, its scope, any pictures.

2

History

The session's last turns, trimmed to fit.

3

Route

The fast model picks one of four routes with one token, and which model answers.

4

Search first

Files, memories and past chats; the web when the question asks for it.

5

Prompt

Built within a budget, stable parts first; a question about your own life lists each source to check.

6

Rounds

The answer model calls tools, up to 8 rounds, the last with none.

7

`dispatch`

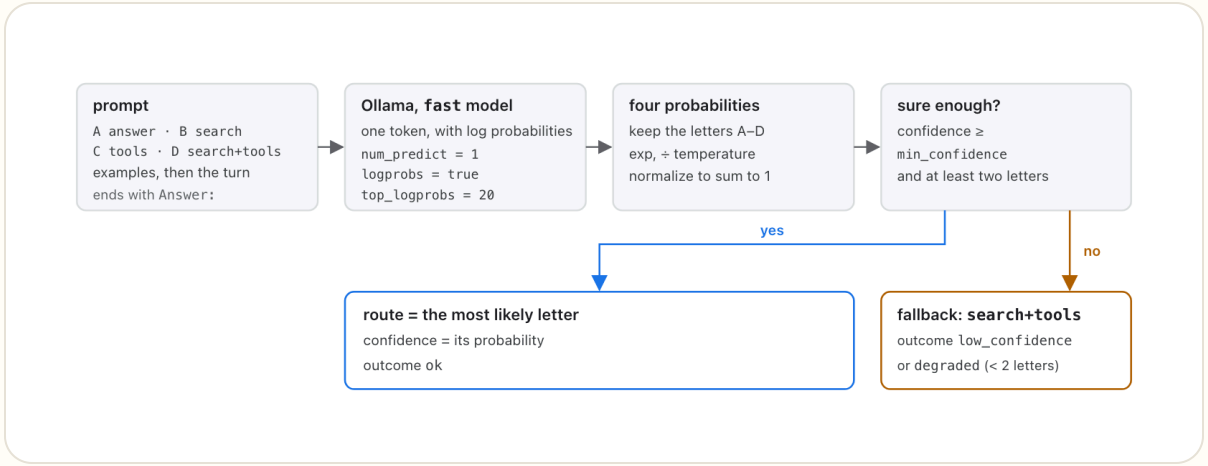
Each call allowed, fenced, approved, logged.

8

Answer

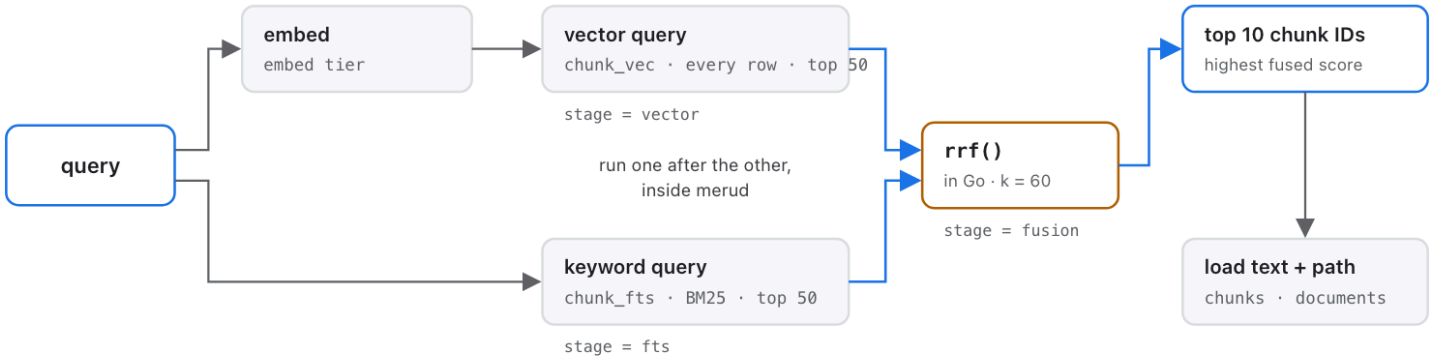
Streams to the client; notes an action no tool took, a file never written, an amount nothing read.

A hand-written loop with no agent framework. A picked skill, such as `mail-research` for Gmail, joins the prompt. `docs/architecture/harness.md` walks it function by function.



A router that writes one letter

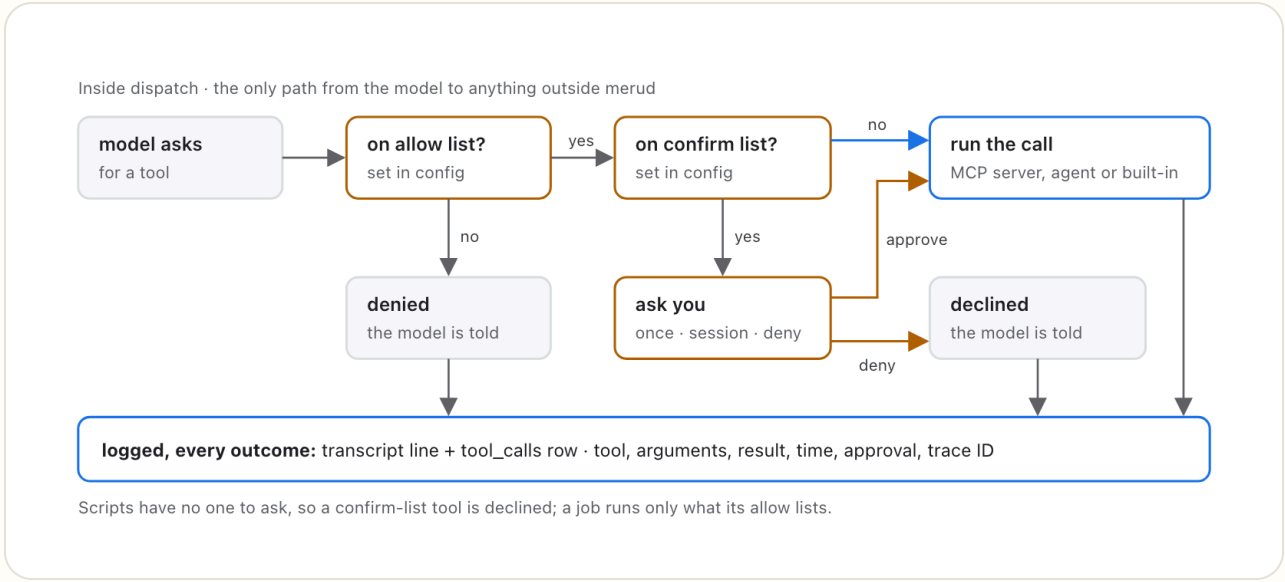
- The fast model writes one of four letters; its log probabilities give a confidence.
- With `answers = "routed"`, the fast model takes a sure greeting, thanks or reword. A short answer goes again to the main model; a question of fact never goes fast.
- `[models] large` answers only on request, with `/large`.



List sizes (50, 50, 10) are constants in internal/retrieve, not config. Each stage reports its time in meru.retrieval.duration.

Search by meaning and by words

- Vectors and BM25 keyword search, 50 each, merged by reciprocal-rank fusion.
- SQLite through `ncruces/go-sqlite3`: no cgo, with vec1 and FTS5 bundled.
- Text and images in one 768-number space with EmbeddingGemma 2.



`dispatch`: the one path to a tool

- MCP servers, A2A agents, `[[commands]]` and 19 built-in tools.
- Allowlist, scope fence, approval (once, session, deny), audit, metrics.
- An incognito chat still gets a `tool_calls` row, with no arguments or result.
- `write_file` puts the local pictures an HTML page names into the page.

CONNECTORS

Meru installs and runs the servers it needs

- A manifest per connector, compiled into `merud`: Obsidian, Google, SearXNG, Ollama and Whisper.
- Every pin is exact: package versions, image digests, SHA-256 per download. A policy test refuses `latest`.
- A supervisor per connector installs on enable, starts on use, stops when idle, and backs off after a crash.

Five kinds

- `stdio` and `http`: MCP servers
- `container`: SearXNG in Docker or Podman
- `service`: a program speaking plain HTTP, such as `whisper-server`
- `dependency`: Ollama, watched, never installed

CONNECTORS

Signing in to MCP servers on the internet

- `auth = "oauth"` on a remote entry. With no `client_id`, Meru registers itself; GitHub and Slack need an app you register.
- The `mcp_sign_in` op runs PKCE (S256) through the MCP Go SDK. A loopback callback takes one GET; the SDK checks `state`.
- The token sits in `~/.meru/oauth/<server>.json` at 0600. No token, code or link reaches a log, a span or an error: each failure becomes a fixed sentence.
- A tools turn sends `sign_in_needed`: a Sign in button in the app, `/signin` in `meru chat`, `meru mcp login`.

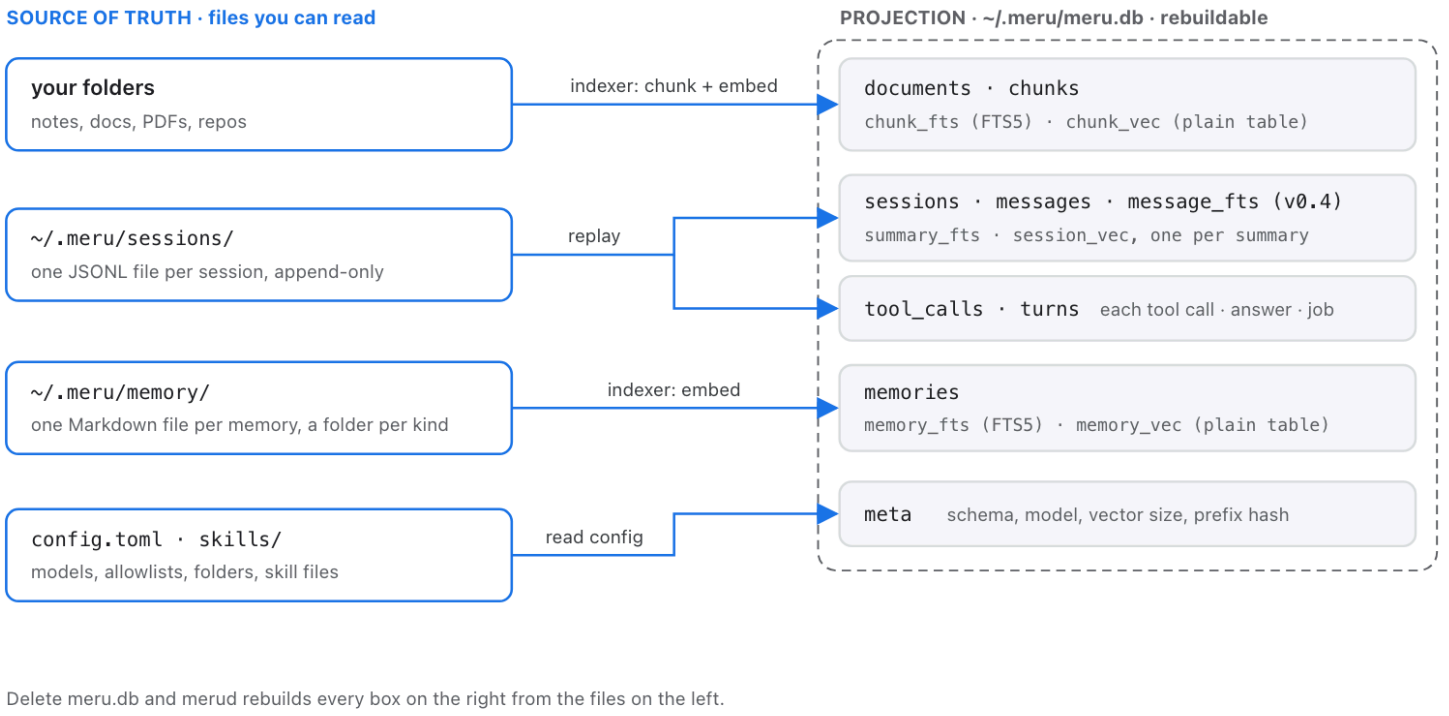
One entry in `config.toml`

```
[[mcp.servers]]
name   = "github"
url    = "https://api.githubcopilot.com/mcp/"
remote = true
auth   = "oauth"
allow  = ["get_me", "search_issues"]

[mcp.servers.oauth]
client_id     = "Ov23li..."
client_secret = "secret:github_client_secret"
issuer        = "https://github.com/login/oauth"
scopes        = ["repo", "read:user"]
```

OAuth requests use https only, with no redirects and no proxy.

STORAGE



Delete meru.db and merud rebuilds every box on the right from the files on the left.

Files are the truth; the database is an index

- JSONL transcripts, Markdown memories, one `config.toml`.
- Delete `meru.db` and Meru rebuilds it.
- A new embedding model drops vectors, keeps text, and re-embeds.

OBSERVABILITY

Measured, and the numbers stay home

- OpenTelemetry metrics and traces for every stage, exported to loopback only; `merud` refuses any other endpoint.
- A `/metrics` page for a Prometheus you run, and a Usage screen in both clients, drawn from `meru.db`.
- Question and answer text stay out of spans unless `capture_content = true`.
- One redactor shows each stored secret as `[secret:<name>]` in `merud.log`, spans and server errors.

Some of the metrics

- `meru.turn.duration` by route, source and outcome
- `meru.route.decisions` : how often the router is unsure
- `meru.tool.calls` by kind, server, tool and outcome
- `meru.retrieval.duration` by stage
- `meru.speech.duration` by model and outcome

Attributes come from small fixed sets: never IDs, paths or text.

A WORKED EXAMPLE

Voice in a day: measure first, then build

The test

- Gemma 4 in Ollama heard well, but at 7.4 GB pushed a model out: about 5 s to a spoken question's first word.
- As a chat model it answered "explain why the sky is blue" instead of writing it down.
- Whisper: 445 to 890 MB in its own process, under half a second for 26 s of speech.

The build, six PRs

- Design in ARCHITECTURE.md first
- CI builds `whisper-server` from pinned source, with a build attestation
- A `service` connector kind, behind a random path per start
- A `transcribe` op, then the microphone in Meru.app

HOW WE WORK

Every change carries its docs and its checks

Contract first

Change `ARCHITECTURE.md`, then carry it into the 300, 200 and 100 pages and their figures.

Explain the code

A PR that changes a package updates its note in `docs/coding-notes/`, or it doesn't merge.

`make check`

Format, vet, staticcheck, race tests, end-to-end tests, five-platform builds, govulncheck, gosec, gitleaks.

Reviewed by personas

The `pr-review` skill reads each PR as security, SRE, Go and architecture reviewers.

Plain prose

Docs, comments and commit messages follow the `writing` skill: short words, active voice.

Releases

`make release` builds, checks and publishes; release notes come from a skill.

CONTRIBUTE

We build in the open, and we're happy to accept contributions

Get going

`git clone https://github.com/aarora79/meru`
`make check`
`go run ./cmd/merud` and `go run ./cmd/meru "hello"`

Read `AGENTS.md`, then `docs/lld.md`, then the coding note for the package you touch. `CONTRIBUTING.md` has the contributor agreement.

From the issue list

- #150 Live transcription
- #87 Third-party MCP servers that Meru installs and heals
- #235 Find AMD cards, not only NVIDIA
- #85 Meru end to end on Windows
- #77 Python examples of `merud` as an agent harness

The first outside contribution, #139, merged in October.



Come build with us

Your assistant, your machine, your data.

We build in the open, and we're happy to accept contributions: **see the issue list** on GitHub.

github.com/aarora79/meru
Scan for the code

github.com/aarora79/meru · aarora79.github.io/meru · Apache License 2.0